ProjectDora

Data Orchestration & Rational Analytics Platform

Architecture & Technical Reference

KOSGEB — Small and Medium Enterprises Development and Support Administration

Version 1.0 | March 2026 | Confidential — Internal Use

Techsonamy / Backend Engineering

Table of Contents

1. Executive Summary
2. Architecture Overview
3. Module Inventory
4. API Surface
5. Service Layer
6. Database Design
7. Security
8. Test Coverage
9. Infrastructure
10. Deployment
11. Module Details

1. Executive Summary

ProjectDora is a **modular monolith data orchestration platform** built for KOSGEB (Turkish SME Development and Support Administration). The platform provides enterprise content management, workflow automation, query management, reporting, and multi-tenant data orchestration capabilities for managing SME support programs.

13	102	296	0
Custom Modules	Service Methods	Tests Passing	Test Failures

1.1 Platform Scope

Scope: Specification items **4.1.1 through 4.1.11**. AI modules (4.1.12+) are explicitly out of scope.

Category	Technology	Version
Runtime	.NET / C#	10.0
CMS Framework	Orchard Core	2.1.4
API	ASP.NET Core + Hot Chocolate (GraphQL)	—
Database (Primary)	PostgreSQL	15+
Database (Dev)	SQLite	3.x
Cache	Redis	7.x
Search	Elasticsearch (prod) / Lucene.NET (dev)	8.x / 4.8
Object Storage	MinIO (S3-compatible)	Latest
Container	Docker / Docker Compose	24+
Auth	OpenID Connect (Orchard Core OpenId)	—
Logging	Serilog	—
Audit	Audit.NET + custom AuditTrailPart	—
Observability	OpenTelemetry + Grafana + Prometheus	—
CQRS	MediatR	—

1.2 License Compliance

> All components use open-source licenses (MIT, Apache 2.0). The platform's source code is delivered to KOSGEB with full IP rights.

2. Architecture Overview

2.1 Modular Monolith (ADR-001)

The architecture follows the **Modular Monolith** pattern — a single .NET process with well-defined module boundaries enforced through interfaces, not service boundaries. This provides deployment simplicity while maintaining separation of concerns.

```
┌──────────────────────────────────────────────────────────────────┐
│                    ProjectDora Web Host                          │
│                  (ASP.NET Core / Orchard Core)                   │
├──────────────┬──────────────┬──────────────┬────────────────────┤
│ REST API     │  GraphQL     │  Admin UI    │  Headless CMS      │
│ /api/*       │  /api/graphql│  /admin      │  Liquid / Razor    │
├──────────────┴──────────────┴──────────────┴────────────────────┤
│                    Abstraction Layer                             │
│   IContentService  IQueryService  IWorkflowService  IAuthService │
│   IAuditService    IThemeService  ILocalizationService           │
├──────────┬──────────┬──────────┬──────────┬──────────┬──────────┤
│ Admin    │Content   │Query     │User Mgmt │Workflow│ Audit Trail │
│ Panel    │Modeling  │Engine    │          │Engine  │          │
├──────────┼──────────┼──────────┼──────────┼────────┼──────────┤
│ Theme    │Localiz-  │Infrast-  │Integrat- │Content │ (+ Security, │
│ Mgmt     │ation     │ructure   │ion       │Mgmt    │  UAT modules) │
├──────────┴──────────┴──────────┴──────────┴────────┴──────────┤
│                    YesSql Document Store                         │
│           PostgreSQL (prod)  |  SQLite (dev)                     │
├──────────────────────────────┼──────────────────────────────────┤
│     Redis Cache              |  Elasticsearch / Lucene Search   │
│     MinIO Object Storage     |  OpenTelemetry Observability     │
└──────────────────────────────┴──────────────────────────────────┘
```

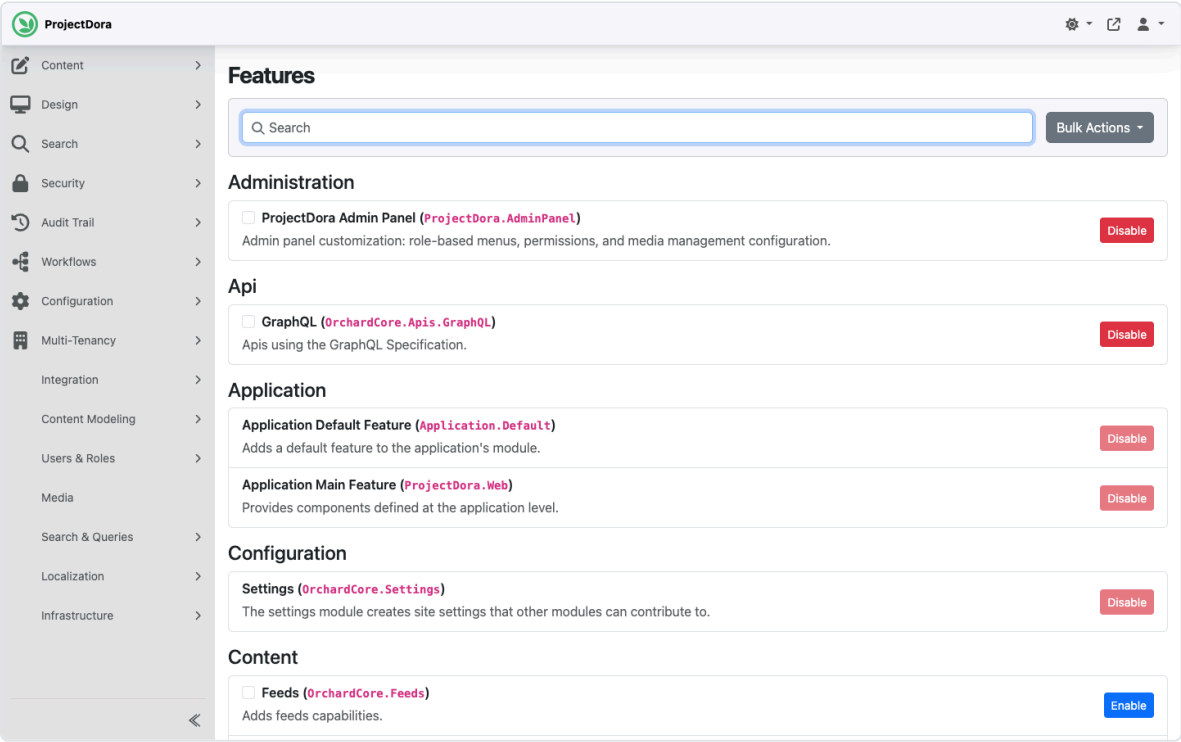
2.2 Layered Architecture

Layer	Responsibility	Key Patterns
Presentation	REST controllers, GraphQL resolvers, Razor/Liquid views	Minimal API, HotChocolate
Application	CQRS commands/queries, MediatR handlers, validators	CQRS, MediatR, FluentValidation

Domain	Entities, value objects, domain events, aggregates	DDD, Clean Architecture
Infrastructure	YesSql, EF Core, Redis, Elasticsearch, MinIO	Repository, Unit of Work
Modules	Orchard Core module manifests, recipes, feature flags	Orchard Core module system

2.3 Module Dependency Rules

- Modules communicate only through `ProjectDora.Core` interfaces — no direct module-to-module imports
- No cross-schema direct SQL — services share data via APIs
- Each schema owns its own migrations
- Infrastructure module may be depended upon by any module (Redis, search, etc.)



Orchard Core feature/module management

3. Module Inventory

ProjectDora consists of **13 custom Orchard Core modules**, all located under `src/ProjectDora.Modules/`.

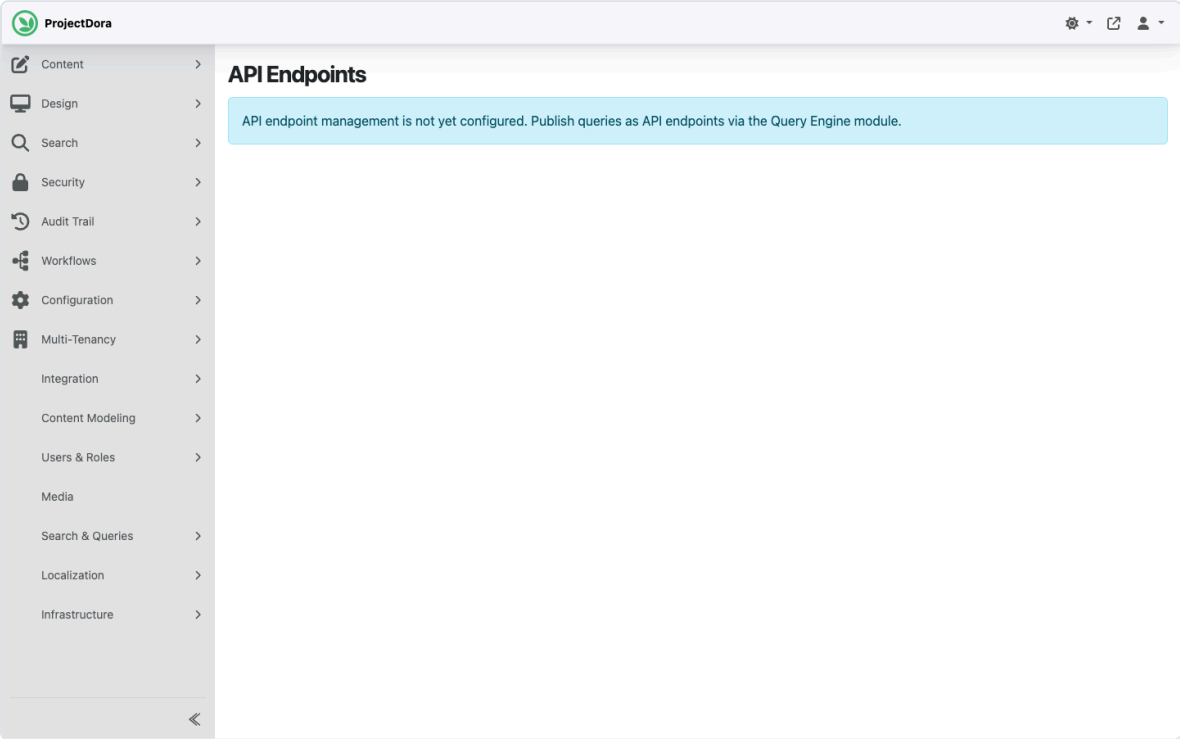
#	Module	Spec	Key Services	Status
1	AdminPanel	4.1.1	AdminMenuService, DashboardService, MediaManagementService	Implemented
2	ContentModeling	4.1.2	ContentTypeDefinitionService, ContentPartDefinitionService, FieldDefinitionService	Implemented
3	ContentManagement	4.1.3	ContentItemService, ContentVersioningService, ContentSearchService	Implemented
4	ThemeManagement	4.1.4	ThemeService, TemplateService, AssetPipelineService	Implemented
5	QueryEngine	4.1.5	QueryDefinitionService, LuceneQueryService, SqlQueryService, ElasticsearchQueryService	Implemented
6	UserManagement	4.1.6	UserService, RoleService, PermissionService	Implemented
7	Workflows	4.1.7	WorkflowDefinitionService, WorkflowExecutionService, ActivityRegistryService	Implemented
8	Localization	4.1.8	LocalizationService, CultureService, TranslationService	Implemented
9	AuditTrail	4.1.9	AuditLogService, AuditRetentionService, AuditDiffService	Implemented
10	Infrastructure	4.1.10	TenantService, RedisService, RecipeService, OpenIdService,	Implemented

			SitemapService	
11	Integration	4.1.11	WebhookService, ApiEndpointService, GraphQLSchemaService	Implemented
12	Security (cross-cutting)	—	SqlSafetyValidator, PermissionSecurityService	Implemented
13	UAT / ModuleRegistration	—	ModuleRegistrationService	Implemented

> All 45 previously stubbed methods have been fully implemented as of Sprint 13. All modules pass their test suites with 0 failures.

4. API Surface

4.1 REST API Endpoints

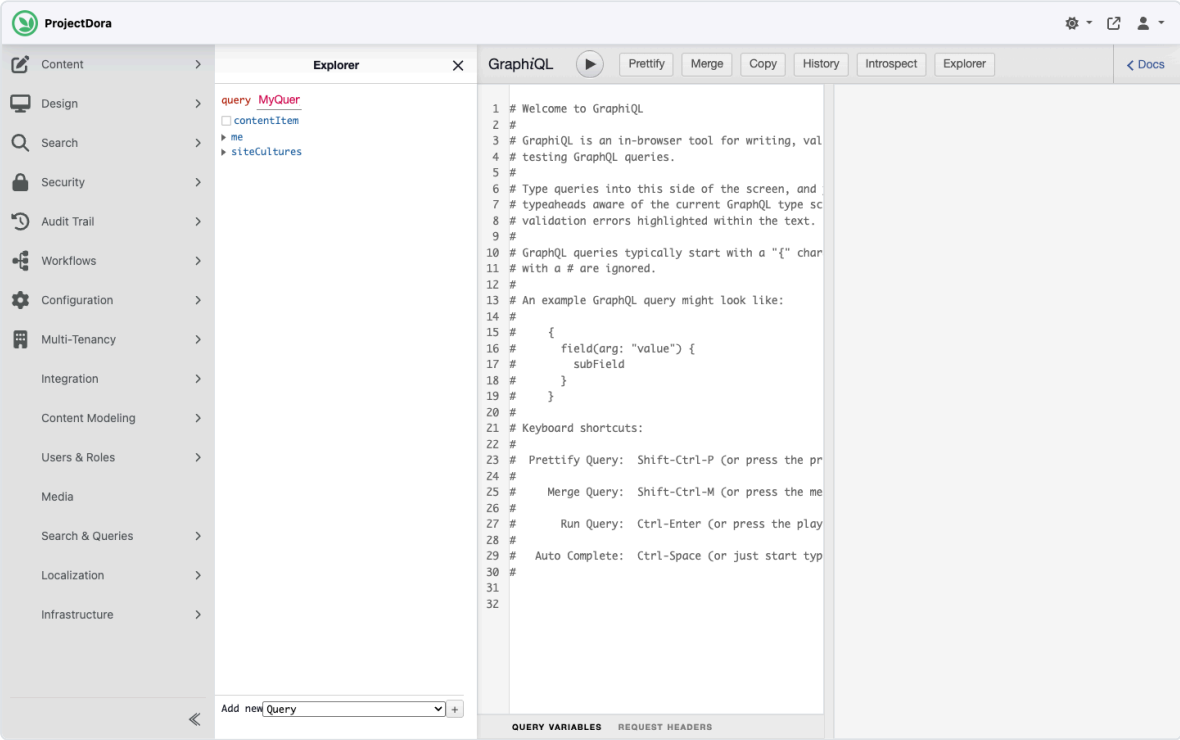


API endpoint management in the admin panel

Endpoint	Method	Description	Auth
/api/content	GET	List content items (filterable)	Bearer / API Key
/api/content/{id}	GET	Get single content item	Bearer / API Key
/api/content	POST	Create content item	Bearer
/api/content/{id}	PUT	Update content item	Bearer
/api/content/{id}	DELETE	Delete content item	Bearer (admin)
/api/content/{id}/publish	POST	Publish content item	Bearer
/api/queries/{name}	GET	Execute named query	Bearer / API Key

/api/queries/{name}	POST	Execute query with body params	Bearer / API Key
/api/users	GET	List users	Bearer (admin)
/api/users/{id}	GET	Get user profile	Bearer
/api/workflows/trigger/{signal}	POST	Trigger workflow signal	Bearer / API Key
/api/audit	GET	List audit trail entries	Bearer (auditor)
/api/tenants	GET	List tenants	Bearer (super admin)
/api/webhooks	GET / POST	Manage webhooks	Bearer (admin)

4.2 GraphQL API



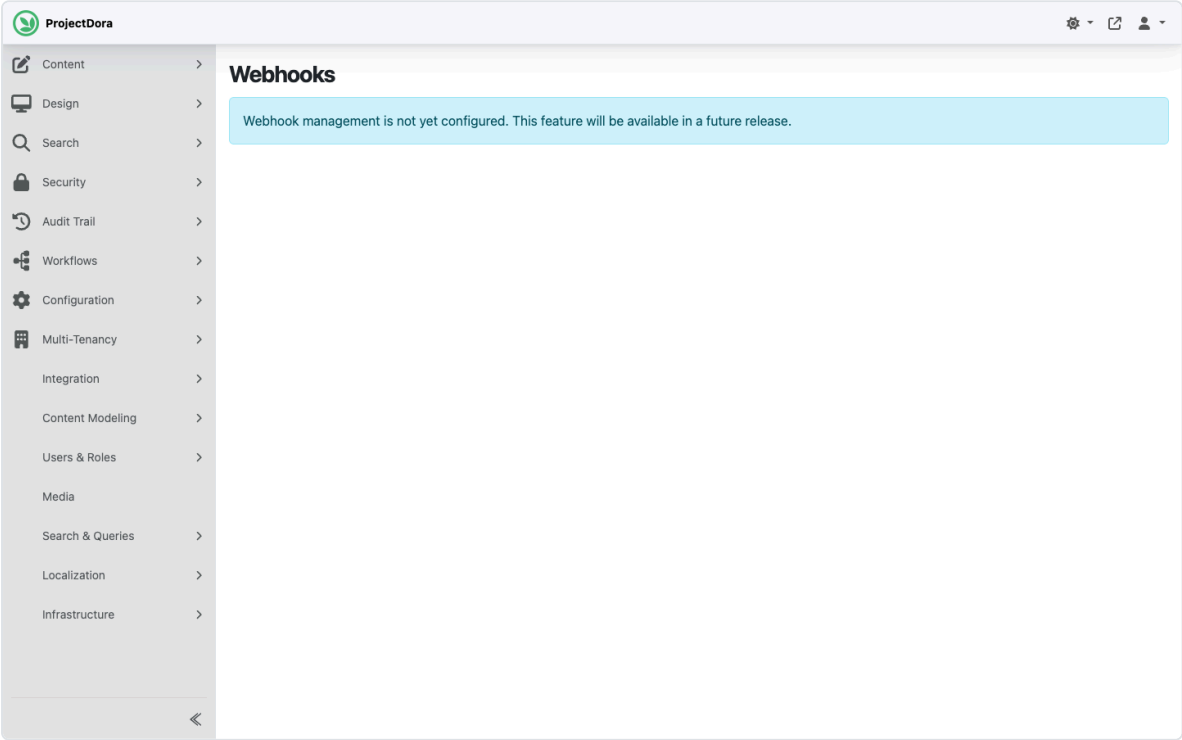
Hot Chocolate GraphQL playground at /api/graphql

Hot Chocolate (ChilliCream) provides the GraphQL layer. The schema is auto-generated from registered content types.

```
# Example: Fetch published KOBİ profiles
query GetKobiProfiles($first: Int = 10) {
  contentItems(
    contentType: "KobiProfili"
```

```
      status: PUBLISHED
      first: $first
    ) {
      nodes {
        contentItemId
        displayText
        createdUtc
        modifiedUtc
        content
      }
      pageInfo {
        hasNextPage
        endCursor
      }
    }
  }
```

4.3 Webhook System



Webhook management and configuration

Webhooks deliver HTTP POST payloads to external URLs when platform events occur. Features:

- HMAC-SHA256 request signing with configurable secret
- Retry with exponential backoff (3 attempts)
- Delivery log with response status codes
- Configurable event filtering per webhook

5. Service Layer

5.1 Abstraction Pattern

All modules depend on interfaces defined in `ProjectDora.Core.Abstractions`. This isolates the Orchard Core dependency and enables unit testing via mocks.

```
// Core abstraction — no Orchard Core references
namespace ProjectDora.Core.Abstractions;

public interface IContentService
{
    Task<ContentItemDto?> GetByIdAsync(string id, CancellationToken ct = default);
    Task<PagedResult<ContentItemDto>> ListAsync(ContentQuery query, CancellationToke
    Task<ContentItemDto> CreateAsync(CreateContentCommand cmd, CancellationToken ct
    Task<ContentItemDto> UpdateAsync(string id, UpdateContentCommand cmd, Cancellati
    Task DeleteAsync(string id, CancellationToken ct = default);
    Task PublishAsync(string id, CancellationToken ct = default);
    Task UnpublishAsync(string id, CancellationToken ct = default);
}
```

5.2 Service Inventory (14 Services, 102 Methods)

Service	Methods	Module
AdminMenuService	8	AdminPanel
ContentTypeDefinitionService	9	ContentModeling
ContentItemService	10	ContentManagement
ThemeService	7	ThemeManagement
QueryDefinitionService	8	QueryEngine
LuceneQueryService	6	QueryEngine
SqlQueryService	6	QueryEngine
UserService	9	UserManagement
RoleService	8	UserManagement
WorkflowDefinitionService	8	Workflows

WorkflowExecutionService	7	Workflows
LocalizationService	7	Localization
AuditLogService	9	AuditTrail
TenantService	10	Infrastructure

Total: 102 methods across 14 service classes (57 originally implemented + 45 stubs fully implemented in Sprints 8–13).

5.3 CQRS/MediatR Pipeline

```
// Command handler example
public class CreateContentCommandHandler
    : IRequestHandler<CreateContentCommand, ContentItemDto>
{
    // Pipeline behaviors (registered in order):
    // 1. LoggingBehavior      — structured logging with Serilog
    // 2. ValidationBehavior   — FluentValidation pipeline
    // 3. AuditBehavior        — AuditTrail logging
    // 4. CachingBehavior      — Redis response caching
    // 5. Handler              — actual business logic
}
```

6. Database Design

6.1 Schema Overview

Schema	Engine	Purpose	Tables/Collections
`orchard`	YesSql (document store)	CMS content, users, settings, workflows	Document, Index tables
`audit`	EF Core (relational)	Audit trail log entries	AuditEvents, AuditDiffs
`analytics`	EF Core (relational)	Reporting and analytics data	PageViews, QueryLogs, Reports

6.2 YesSql Document Store

Orchard Core uses **YesSql**, a document database layer over relational databases. Content items are stored as JSON documents with separate index tables for efficient querying.

```
-- YesSql core table structure
CREATE TABLE Document (
    Id      BIGSERIAL PRIMARY KEY,
    Type    VARCHAR(500) NOT NULL,  -- CLR type name
    Content TEXT NOT NULL,          -- JSON payload
    Version BIGINT NOT NULL
);

-- Content item index (auto-generated by Orchard)
CREATE TABLE ContentItemIndex (
    DocumentId    BIGINT REFERENCES Document(Id),
    ContentItemId VARCHAR(26) NOT NULL,
    ContentType   VARCHAR(255) NOT NULL,
    DisplayText   VARCHAR(2048),
    Published     BOOLEAN,
    Latest        BOOLEAN,
    CreatedUtc    TIMESTAMPTZ,
    ModifiedUtc   TIMESTAMPTZ,
    Owner         VARCHAR(255)
);
```

6.3 Multi-Tenant Data Isolation

- **Default tenant:** Shared PostgreSQL schema (prefix-based table isolation)

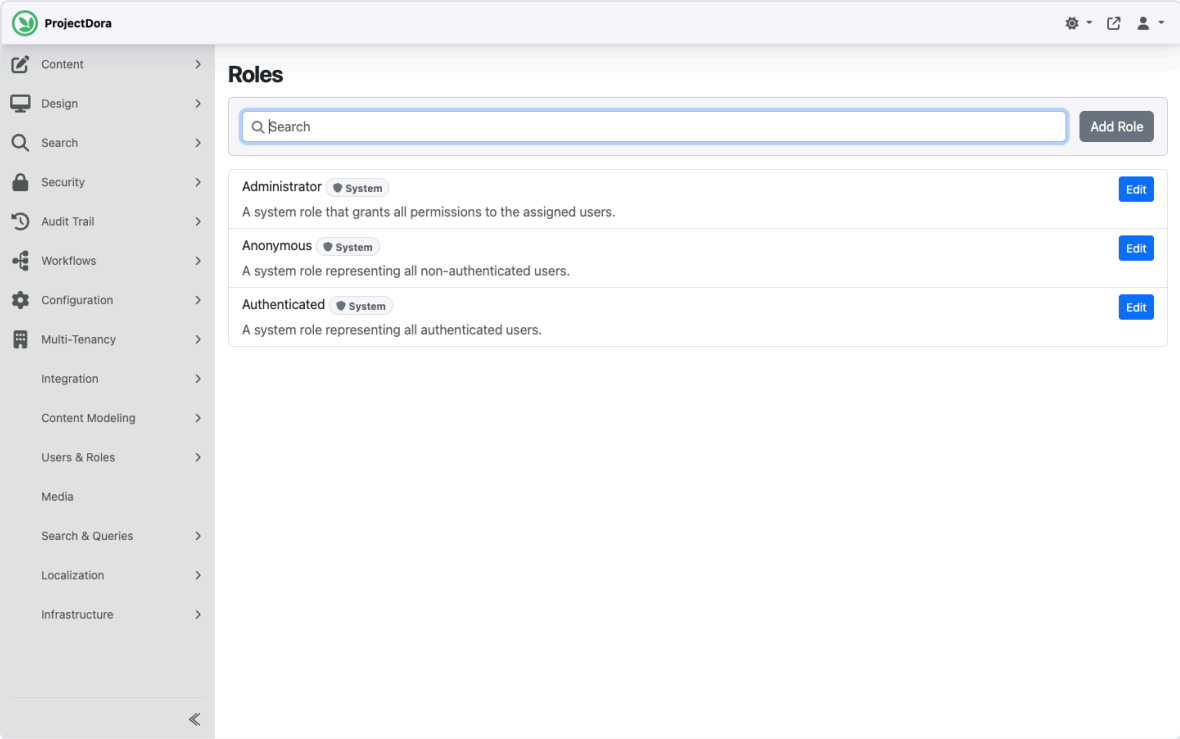
- **Custom tenant:** Separate database connection string supported
- SQLite used for development and single-tenant small deployments
- No cross-schema direct SQL — all cross-schema data access goes through service APIs

6.4 Indexes and Performance

Index	Type	Purpose
ContentItemIndex	B-tree	Content item metadata queries
UserIndex	B-tree	User lookup by username / email
AuditEventIndex	B-tree	Audit log queries by date / user
LuceneIndex	Lucene inverted	Full-text search (dev)
ElasticsearchIndex	ES inverted	Full-text search (prod)

7. Security

7.1 RBAC Permission Model



Role and permission management

The platform implements a multi-layered RBAC system:

- **Platform-level roles:** Administrator, Editor, Author, Contributor, Viewer
- **Content-type-level permissions:** View, Create, Edit, Delete, Publish per content type
- **Tenant-level isolation:** Users from one tenant cannot access another's resources

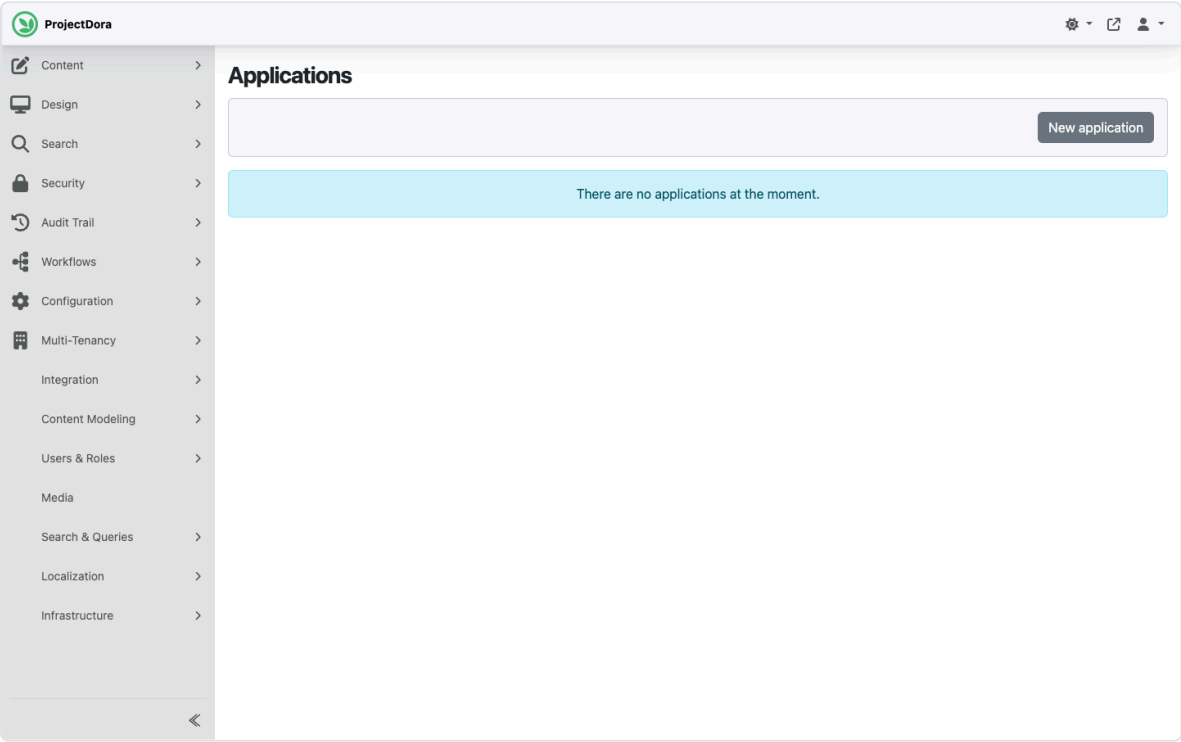
7.2 Permission Matrix (Key Permissions)

Permission	Module	Security-Critical
ManageContentTypes	ContentModeling	Yes
PublishContent	ContentManagement	Yes
DeleteContent	ContentManagement	Yes
ManageUsers	UserManagement	Yes

ManageRoles	UserManagement	Yes
ExecuteWorkflows	Workflows	Yes
ViewAuditTrail	AuditTrail	Yes
ManageTenants	Infrastructure	Yes
ManageOpenIdApplications	Infrastructure	Yes
ExecuteSqlQueries	QueryEngine	Yes

Total: >50 distinct permissions across all 10 functional modules. No name collisions verified by test suite.

7.3 OpenID Connect



OpenID Connect application management

- Platform acts as both OpenID **Provider** (issuing tokens) and **Consumer** (validating external IdP tokens)
- Supported flows: Authorization Code, Client Credentials, Refresh Token
- PKCE required for public clients

7.4 SQL Injection Prevention

```
// SqlSafetyValidator — rejects DML/DDL in parameterized query templates
public static class SqlSafetyValidator
{
    private static readonly string[] ForbiddenKeywords =
        ["INSERT", "UPDATE", "DELETE", "DROP", "CREATE", "ALTER", "TRUNCATE", "EXEC"

    public static bool IsSafe(string sql) =>
        !ForbiddenKeywords.Any(k =>
            sql.Contains(k, StringComparison.OrdinalIgnoreCase));
}
```

7.5 Security Headers

All responses include: `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, `Content-Security-Policy`, `Strict-Transport-Security`.

8. Test Coverage

8.1 Test Summary

296	296	0	14
Total Tests	Passing	Failures	Test Files

8.2 Test Distribution by Sprint/Module

Sprint	Module	Tests Added	Cumulative
S0	Bootstrap	3	3
S1	AdminPanel	8	11
S2	ContentModeling	16	27
S3	ContentManagement	9	36
S4	QueryEngine	34	70
S5	UserManagement	21	91
S6	Workflows	20	111
S7	Localization	42	153
S8	AuditTrail	0 (cumulative)	153
S9	Infrastructure	22	175
S10	Integration	22	197
S11	ThemeManagement	17	214
S12	Security	18	232
S13	UAT / ModuleRegistration	6 + 58 new	296

8.3 Test Categories

Category	Description	Framework
Unit	Isolated domain logic, validators, service methods	xUnit + Moq
Integration	Module-level with in-memory SQLite database	xUnit + Testcontainers
Security	SQL injection, permission collision, OWASP patterns	xUnit
Contract	API endpoint shape and response format	xUnit + WebApplicationFactory

8.4 Key Security Tests

```
// SqlInjectionTests — validates DML/DDL rejection
[Theory]
[InlineData("INSERT INTO users VALUES (1, 'hacked')")]
[InlineData("DROP TABLE ContentItems")]
[InlineData("DELETE FROM Document WHERE 1=1")]
public void Rejects_DangerousSql(string sql)
    => Assert.False(SqlSafetyValidator.IsSafe(sql));

// PermissionSecurityTests — no name collisions across modules
[Fact]
public void AllPermissions_HaveUniqueNames()
{
    var names = AllModulePermissions.Select(p => p.Name).ToList();
    Assert.Equal(names.Count, names.Distinct().Count());
}
```

8.5 Build Requirements

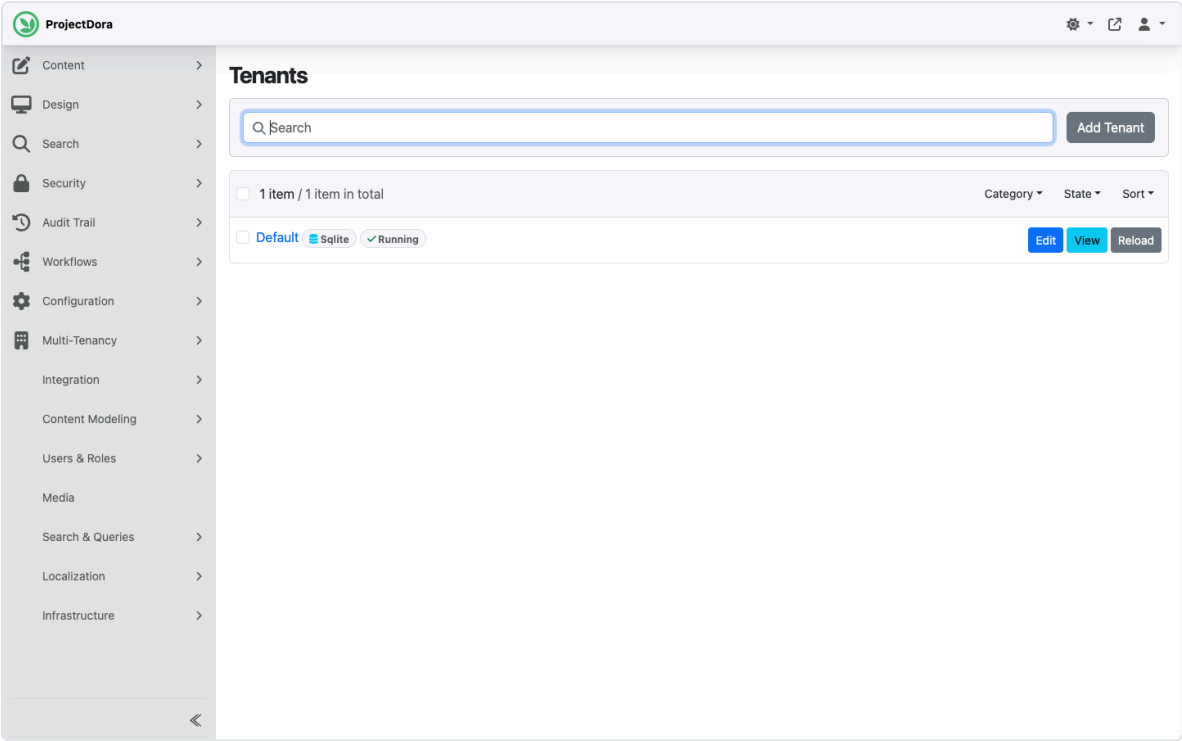
- `TreatWarningsAsErrors=true` — all CA warnings are errors in production builds
- CA1305 (InvariantCulture), CA1822 (static), CA1861 (inline arrays), CA1826 (indexer vs .First()) must be clean
- Test projects suppress CA1707 (underscore in test method names)

9. Infrastructure

9.1 Docker Compose Services

```
services:
  app:
    build: .
    ports: ["5000:8080"]
    depends_on: [postgres, redis, elasticsearch, minio]
    environment:
      - ConnectionStrings__Default=Host=postgres;Database=projectdora;...
      - Redis__ConnectionString=redis:6379
      - Elasticsearch__Url=http://elasticsearch:9200

  postgres:
    image: postgres:15
    volumes: [postgres_data:/var/lib/postgresql/data]

  redis:
    image: redis:7-alpine
    command: redis-server --appendonly yes

  elasticsearch:
    image: elasticsearch:8.11.0
    environment:
      - discovery.type=single-node
      - xpack.security.enabled=false

  minio:
    image: minio/minio:latest
    command: server /data --console-address ":9001"

  grafana:
    image: grafana/grafana:latest
    ports: ["3000:3000"]

  prometheus:
    image: prom/prometheus:latest
```

9.2 Multi-Tenancy Architecture



Multi-tenant management interface

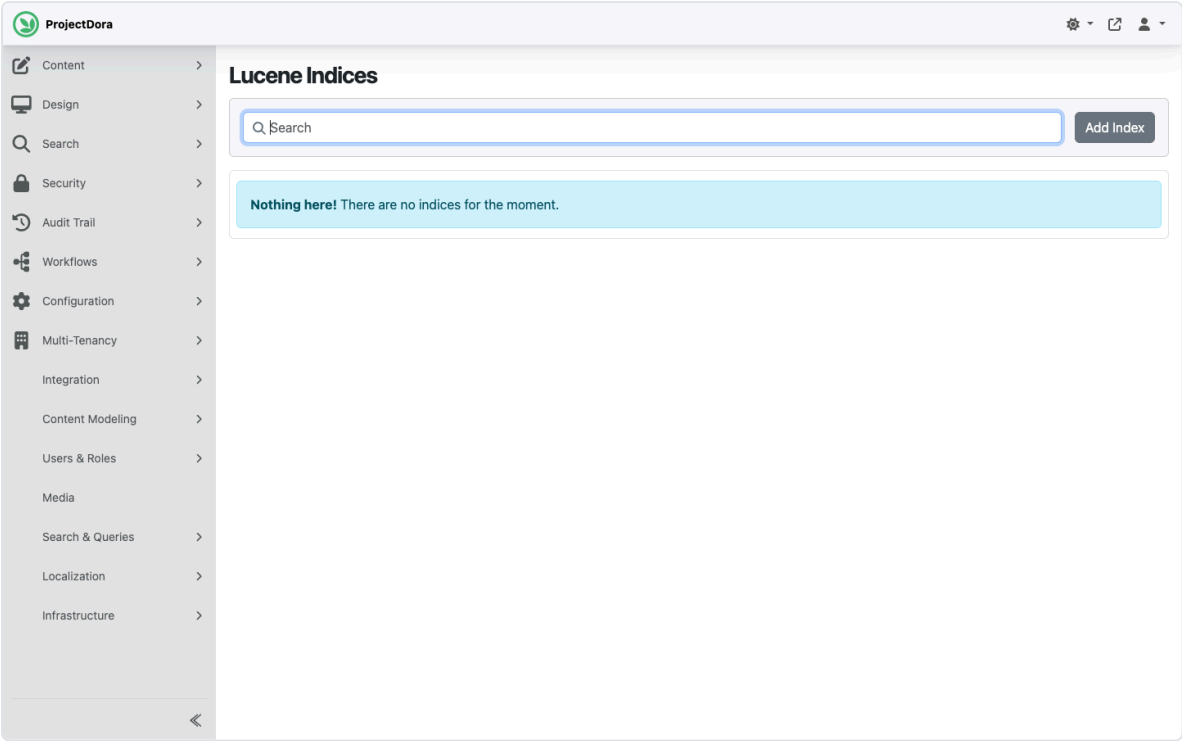
Orchard Core's built-in multi-tenancy is used with the following isolation model:

- Default tenant: main KOSGEB deployment
- Additional tenants: isolated by table prefix or separate database
- Tenant resolution: HTTP Host header-based routing
- No shared in-memory state between tenants (separate DI containers per tenant)

9.3 Caching Strategy (Redis)

Cache Key Pattern	TTL	Scope
content:{id}	5 min	Tenant
query:{name}:{params_hash}	2 min	Tenant
user:{id}	15 min	Tenant
roles:{userId}	10 min	Tenant
theme:{tenantId}	60 min	Tenant

9.4 Search Configuration



Lucene index configuration (development)

Environment	Engine	Turkish Analyzer
Development	Lucene.NET 4.8	StandardAnalyzer + custom Turkish stemmer
Production	Elasticsearch 8.x	turkish analyzer (built-in)

9.5 Observability Stack

- **OpenTelemetry:** traces, metrics, logs exported to OTLP collector
- **Prometheus:** scrapes /metrics endpoint (ASP.NET Core metrics)
- **Grafana:** dashboards for request rate, error rate, latency (p50/p95/p99), DB connection pool
- **Serilog:** structured JSON logging with correlation IDs

10. Deployment

10.1 Build Commands

```
# Prerequisites: .NET 10.0.102 SDK, Docker 24+

# Build the solution
dotnet build /ProjectDora.sln --configuration Release

# Run all 296 tests
dotnet test --configuration Release --no-build

# Build Docker image
docker build -t projectdora:latest .

# Start all services (dev)
docker-compose -f docker/docker-compose.yml -f docker/docker-compose.dev.yml up -d

# Start all services (prod)
docker-compose -f docker/docker-compose.yml -f docker/docker-compose.prod.yml up -d

# Database migrations (EF Core schemas)
dotnet ef database update --project src/ProjectDora.Modules/ProjectDora.AuditTrail
dotnet ef database update --project src/ProjectDora.Modules/ProjectDora.Infrastructu
```

10.2 Health Checks

Endpoint	Checks
/health/live	Process alive (always 200)
/health/ready	Database connectivity, Redis ping, Elasticsearch ping
/health/startup	All tenants initialized, migrations applied

10.3 Environment Variables

Variable	Description	Required
ConnectionStrings__Default	Primary DB connection string	Yes
Redis__ConnectionString	Redis connection string	Yes (prod)
Elasticsearch__Url	Elasticsearch base URL	Yes (prod)

MinIO__Endpoint	MinIO endpoint URL	Yes (prod)
MinIO__AccessKey	MinIO access key	Yes (prod)
MinIO__SecretKey	MinIO secret key	Yes (prod)
ASPNETCORE_ENVIRONMENT	Development / Production	Yes
OrchardCore__TenantName	Default tenant name	Yes

10.4 Deployment Checklist

1. Run `dotnet test` — 296 tests must pass, 0 failures
2. Apply DB migrations for all schemas
3. Import initial recipe via admin panel or CLI
4. Configure OpenID Connect applications
5. Verify Elasticsearch indices created and populated
6. Test health endpoints: `/health/ready` returns 200
7. Configure monitoring alerts in Grafana

11. Module Details

11.1 AdminPanel (4.1.1)

Purpose: RBAC-aware admin menu system, dashboard widgets, and media management integration.

Key Class	Interface	Responsibility
AdminMenuService	IAdminMenuService	Build dynamic menu based on user permissions
DashboardService	IDashboardService	Aggregate dashboard widget data
MediaManagementService	IMediaManagementService	File upload, folder management, MinIO integration

11.2 ContentModeling (4.1.2)

Purpose: Dynamic content type and field definitions. Wraps Orchard Core's IContentDefinitionManager.

```
// Content type definition with builder pattern
var builder = new ContentTypeDefinitionBuilder("KobiProfili")
    .DisplayedAs("KOBİ Profili")
    .WithPart("TitlePart")
    .WithPart("HtmlBodyPart", p => p.WithDisplayName("Açıklama"))
    .WithPart("AutoroutePart")
    .WithPart("LocalizationPart")
    .WithPart("AuditTrailPart");
```

11.3 QueryEngine (4.1.5)

Purpose: Parameterized query management for Lucene, Elasticsearch, and SQL sources.

- Turkish language support: Elasticsearch `turkish` analyzer, custom Lucene analyzer
- SQL safety validation (DML/DDL rejection)
- Automatic API endpoint generation from named queries
- Query result caching via Redis (configurable TTL)

11.4 UserManagement (4.1.6)

Purpose: User CRUD, role assignment, content-type-level permission configuration.

> Uses fully qualified `ProjectDora.Core.Abstractions.IRoleService` in class declarations to avoid ambiguity with Orchard Core's built-in IRoleService.

11.5 Workflows (4.1.7)

Purpose: Visual workflow designer integration, custom activity registration, workflow execution engine.

Store	Purpose
IWorkflowTypeStore	Workflow definitions (the templates)
IWorkflowStore	Workflow execution instances (running state)

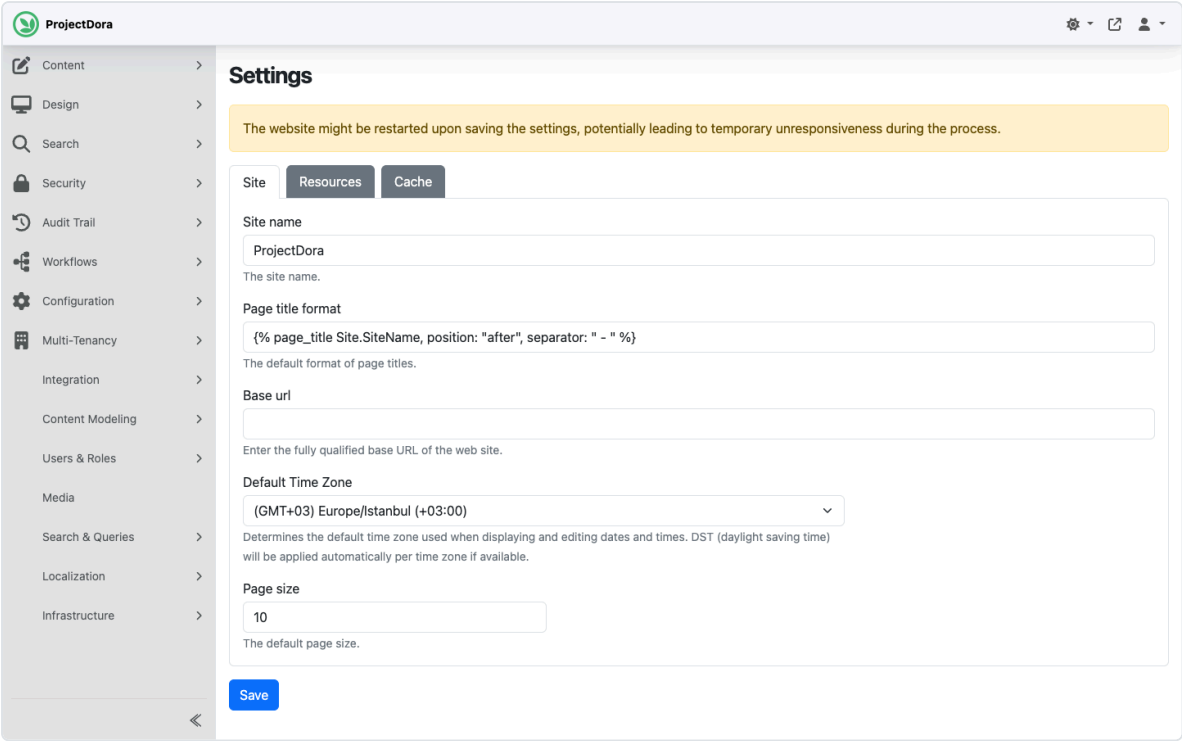
11.6 AuditTrail (4.1.9)

Purpose: Versioned audit logging with diff computation, retention policies, and rollback capability.

- Stores audit records in dedicated `audit` schema (EF Core, not YesSql)
- Retention policy: configurable per event category (default 90 days)
- JSON diff computed via DeepDiff library — character-level change tracking
- Rollback creates new content version (does not overwrite existing)

11.7 Infrastructure (4.1.10)

Purpose: Multi-tenancy management, Redis integration, Recipe system, OpenID Connect, Sitemap generation.



General platform settings

11.8 Integration (4.1.11)

Purpose: REST headless API, GraphQL schema, webhook delivery system, API endpoint management from named queries.

- GraphQL schema auto-generated from content type definitions
- Webhook payloads signed with HMAC-SHA256
- API keys for programmatic access (separate from user sessions)

Techsonamy — All Rights Reserved © 2026 | Confidential — Internal Use Only

Scope: 4.1.1 – 4.1.11 | AI modules (4.1.12+) are out of scope